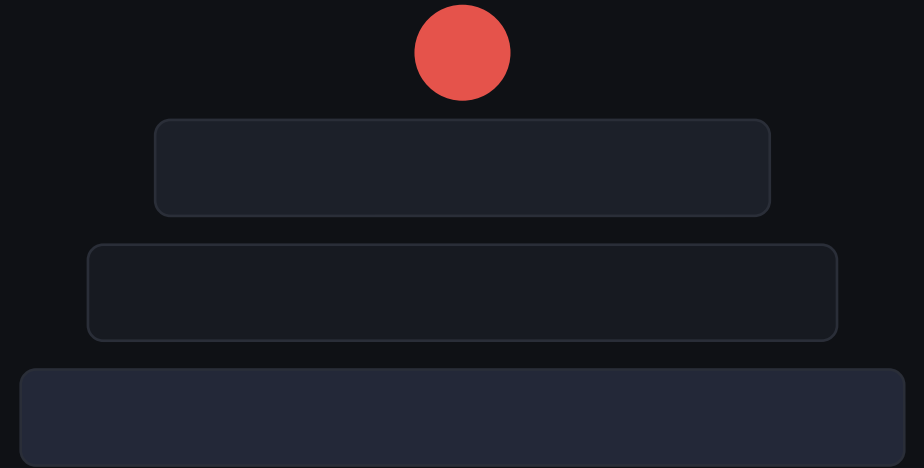


MULTI-AGENT AI SRE PLATFORM

Autonomous incident investigation for Kubernetes

Alerts fire, the platform investigates on its own, produces a grounded root-cause report and an execution-ready runbook — and a human approves anything that touches the cluster.

AI/ML Bootcamp Capstone





SRE toil doesn't scale with headcount

01

Alerts outpace attention

A cascading outage can fire dozens of alerts in minutes. Someone still has to open Grafana, Jaeger, and kubectl, by hand, for every one.

02

Root cause lives across three tools

Metrics say what broke. Traces say where. Kubernetes events say why. No single dashboard correlates all three under time pressure.

03

Tribal knowledge doesn't transfer

The engineer who remembers last quarter's near-identical incident isn't always the one on call this time.



One copilot, two ways in, one approval gate

Pull: a human asks the copilot a question over chat. Push: Alertmanager posts a firing alert and the copilot investigates it with nobody watching. Both converge on the same LangGraph agent, the same tool set, and the same human-in-the-loop approval before anything touches the cluster.



19

COPILOT TOOLS



4

DOMAIN AGENTS



6

MILESTONES SHIPPED



193

AUTOMATED TESTS

Stack: OpenTelemetry Demo on Kind · Prometheus / Alertmanager / Grafana · Jaeger + OTel Collector · Python 3.14, LangGraph, FastAPI



Six layers, one approval gate

Front ends

CLI (chat.py) · Web UI — Chat / Timeline / Ops Hub

Copilot

SRE Copilot — LangGraph ReAct agent, 19 tools

Coordinator + Analysis

Investigation graph (RCA + runbook) · Alert correlator · anomaly detector · incident search

Agents

Metrics · Trace · Kubernetes · Alert

Clients

Prometheus · Jaeger · Kubernetes API · Alertmanager

Kind cluster

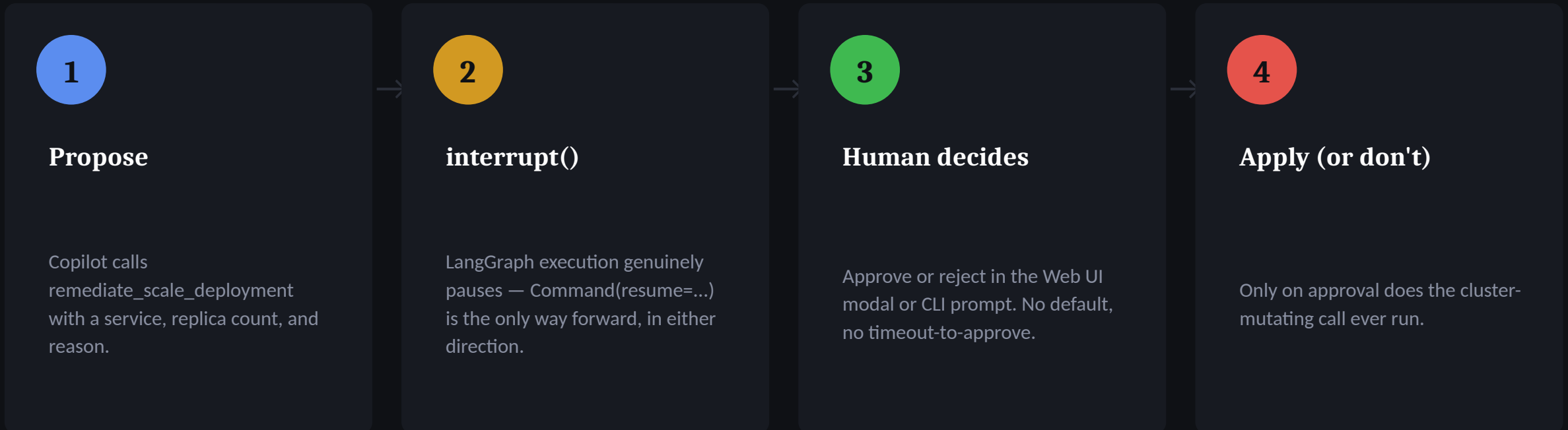
OTel Demo app · Prometheus/Grafana/Alertmanager · Jaeger

Human approval is the only path from read to write — every layer above it is read-only.



The write path is unreachable without a human

Every layer of the platform is read-only except one: `scale_deployment`. It's gated by a real pause in the LangGraph execution — not a confirmation prompt the agent could talk its way past.



Also requires prior evidence-gathering in the same conversation — a purely speculative remediation call is blocked before a human ever sees it.



Found, then closed: five real vulnerabilities

VULNERABILITY

FIX



Approval gate bypassable over the network

server.py bound 0.0.0.0 with no auth — anyone reaching the port could approve a pending remediation.



Bind 127.0.0.1 by default + Google OIDC on every human-facing route.



Stored XSS into the approval flow

Alert-derived fields interpolated into innerHTML unescaped — an attacker-controlled alertname could execute script.



Every render path rebuilt with textContent, never innerHTML.



PromQL injection

Service names from LLM tool calls (sourced from untrusted alert annotations) were f-string'd into queries.



PromQL string-literal escaping at every interpolation point.



Unbounded background investigations

A 40-alert cascade could fire 40 concurrent multi-LLM-call investigations with no cap.



Bounded semaphore (default 3); excess incidents queue instead of piling on.



Import architecture / package hygiene

20 scattered sys.path hacks, an unimportable ai-platform directory name, a stdlib name collision.



Real installable package, 37 files converted to package-qualified imports.



Real auth, and defense-in-depth for data leaving the cluster

Google OIDC sign-in

- Replaced a shared-secret API key with per-user Google sign-in — a signed, httponly session cookie, not a secret every user shares.
- Optional ALLOWED_GOOGLE_EMAILS / ALLOWED_GOOGLE_DOMAIN allowlists layer authorization on top of authentication.
- The one exception: Alertmanager's webhook keeps a scoped Bearer-token secret, since a machine can't complete an OAuth redirect.
- Known gap, stated plainly: no per-role authorization yet — every allowlisted user can also approve a remediation.

PII / secret redaction

- Pod logs, K8s events, and RCA text reach a third-party LLM and get persisted to SQLite — nothing upstream guaranteed no PII in that path.
- Regex scrub for emails, SSNs, card numbers, phone numbers, API keys — applied twice: at the source client, and again as a backstop when the copilot serializes results.
- IPs and hostnames deliberately left alone — core SRE signal, not personal data.
- Stated honestly as a floor, not a guarantee: pattern-based, will miss free-text PII.



Three tabs, one incident data source



Operations hub

Default landing tab. Incident count, resolution rate, median time-to-mitigate, pending approvals, a daily volume/outcomes chart, and recent incidents — all aggregated server-side from the same data the other two tabs read.



Timeline

Every auto-triaged incident, grouped by day, oldest to newest — the sequence of a cascading outage reads top to bottom the way it actually happened.



Chat

Conversational copilot with a live tool-call trace and the approval modal — ask a question, or click into any incident from the other two tabs to see its investigation.



What we'll walk through

1

Trigger

`kubect! scale deployment product-catalog --replicas=0` — a real failure in the live cluster.

2

Auto-triage

Alertmanager posts to the webhook; the copilot picks it up with nobody asking, no polling.

3

Investigate

Metrics, trace, and Kubernetes agents gather evidence in parallel; the coordinator correlates it.

4

Root cause + runbook

An LLM produces a grounded RCA and an execution-ready remediation runbook.

5

Approve

The proposed fix parks on `awaiting_approval` until a human clicks Approve — in the Web UI or CLI.

6

Resolve

Operations Hub's KPIs and the Timeline update live — resolution rate, time-to-mitigate, the works.

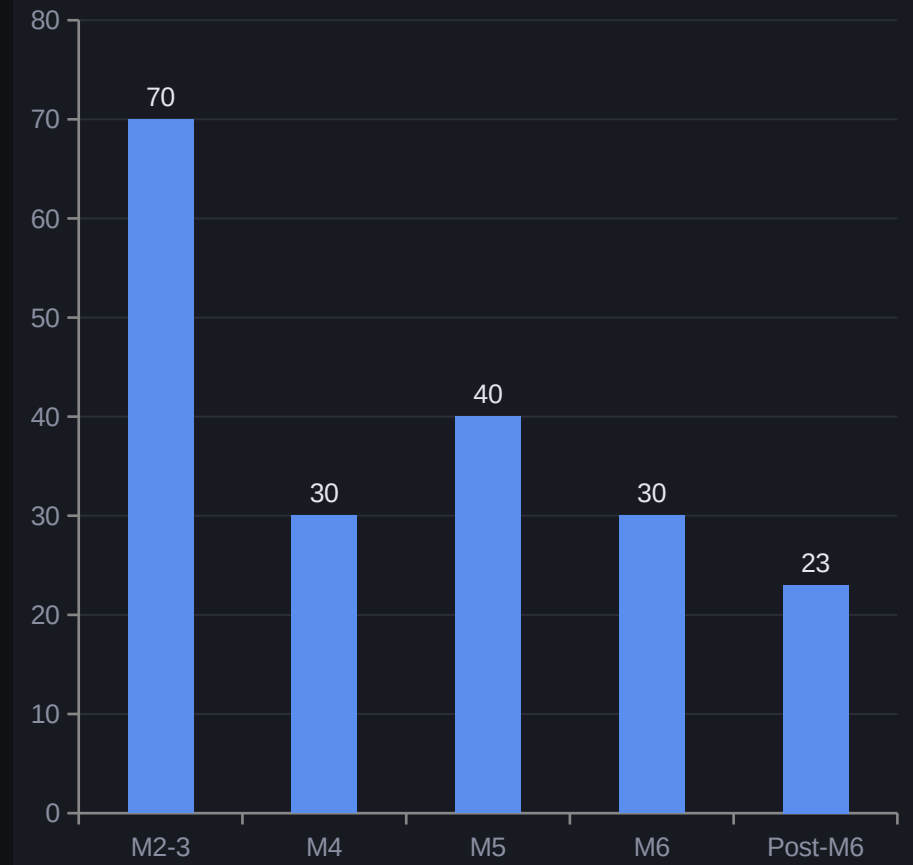


Every milestone shipped with tests, not just code

The full suite is mocked — no cluster, API key, or network required to run it. Live-cluster smoke scripts exist separately and are deliberately excluded from the offline run.

- `uv run pytest` — 193 tests, zero cluster dependency
- Each of the four subpackages owns its own tests/ directory
- Security fixes shipped with regression tests, not just patches
- Operations Hub KPI math independently unit-tested against seeded incident records

Test count by milestone (approx.)





Stated plainly, not hidden

Accepted trade-offs

- No per-role authorization — every allowlisted user can also approve a remediation.
- PII redaction is pattern-based, not a compliance-grade classifier.
- OAuth client secrets and the session key still live in plaintext .env.
- Token refresh isn't implemented — sessions simply expire after 8 hours.

On the roadmap

- Slack / Microsoft Teams integration
- MCP server support
- Knowledge base integration
- True embedding-based incident search (current search is TF-IDF keyword overlap)

Questions

Multi-Agent AI SRE Platform — thank you.

